NKT
Tag 1: Grundlagen, Datentypen und Methoden

Tobias Wrede, Alexa Katharina-Ronja Brockmöller

8. März 2026

Gliederung

1. Organisation
2. Basics
   - Variablen
   - Datentypen
   - Arrays
3. Kontrollstrukturen
   - Verzweigungen
   - Schleifen
   - Idiomatisch
4. Methoden
   - Methodenaufruf
   - Main Methode
   - Rekursion
5. Classpath und Package-Pfad

Gliederung

1 Organisation

2 Basics
- Variablen
- Datentypen
- Arrays

3 Kontrollstrukturen
- Verzweigungen
- Schleifen
- Idiomatisch

4 Methoden
- Methodenaufruf
- Main Methode
- Rekursion

5 Classpath und Package-Pfad

Link zum Repository

- **GitHub für dieses Tutorium** (hier findet ihr auch diese Präsentation):
  `https://github.com/alexabrock/Programmierung-Nachklausurtutorium`

Scan Me

Disclaimer - Quellen dieser Unterlagen

Heinrich Heine Universität Düsseldorf

Wichtiger Hinweis zu den Inhalten

- Diese Materialien basieren direkt auf den Vorlesungsunterlagen von Markus Brenneis
- Die Folien basieren auf den Folien von Sven Hoops und Jonas Goudarzi

Wie für die Klausur lernen (offizielle Version)

hhu
Heinrich Heine
Universität Düsseldorf

Nur Altklausuren lernen = Schiefgehen!

- Altklausuren sind **kein Ersatz** für intensive Mitarbeit im Semester
- Klausuren prüfen **tieferes Verständnis** ab, nicht nur Auswendiglernen
- **Alle Wochenblätter sind relevant**, auch die letzten Blätter

Richtige Lernstrategie

- Von jedem Woche die **Wichtigen Begriffe** erklären können & die **Lernziele** beachten
- **Alle Aufgaben selbstständig** bearbeiten
- **Anderen den Stoff erklären** (Lerngruppen!)
- Muster in den Altklausuren erkennen und verstehen, wie man die Aufgaben angeht

Reflexion von Fehlern

Heinrich Heine Universität Düsseldorf

Hinweise für die Fehleranalyse

- Schaut euch eure korrigierte 1. Klausur/ Probeklausur noch einmal genau an.
- Fragt euch: „Warum habe ich diese Aufgabe nicht richtig gelöst?“
- Überlegt euch, was schiefgelaufen ist und wie ihr es beim nächsten Mal vermeiden könnt.
- Wer die Fehler versteht, lernt am meisten.

Nachklausurtutorium Programmierung

Heinrich Heine Universität Düsseldorf

Zeitraum: Montag den 09.03.2026 bis Freitag den 13.03.2026
Zeit: Täglich 10:30 bis 14:00 Uhr
Ort: Hörsaal 5H

Gliederung

1. Organisation
2. Basics
   - Variablen
   - Datentypen
   - Arrays
3. Kontrollstrukturen
   - Verzweigungen
   - Schleifen
   - Idiomatisch
4. Methoden
   - Methodenaufruf
   - Main Methode
   - Rekursion
5. Classpath und Package-Pfad

Kompilieren

- Kompilieren
  - Quellcode wird in Bytecode übersetzt
  - Überprüfung, ob Regeln der Sprache eingehalten wurden
- Java Programm kompilieren und ausführen
  - java Programmname.java

Variablendeklaration

Heinrich Heine Universität Düsseldorf

In Java benutzen wir häufig Variablen, um Daten zu speichern. Um eine Variable anzulegen benötigen wir einen Datentypen und einen Namen. Das Anlegen einer Variable heißt Deklaration:

```
int f;
float g;
String msg;
```

primitive Variablen, die nur deklariert wurden, kriegen einen Standardwert zugewiesen. Bei *int* ist dieser beispielsweise 0

Variableninitialisierung

Bei der Initialisierung von Variablen weisen wir diesen nun einen Wert zu:

```
int f = -2;
float g = 42.69;
String msg = "Hello World";
```

Referenzen gelten hier auch als Werte, die wir zuweisen können:

```
String importantMessage = msg + "!";
```

Primitive Datentypen

Heinrich Heine Universität Düsseldorf

Welche primitiven Datentypen gibt es?

- boolean
- byte
- char
- short
- int
- long
- float
- double

Nicht primitive Datentypen

Heinrich Heine Universität Düsseldorf

- Objektreferenzierung
  - speichert anstelle des Objektes nur die Referenz auf dem Stack
  - erlaubt die Einschränkung von Zugriff auf Attribute und Variablen des Objektes
  - ist häufig nötig, da Stack Platz sehr begrenzt
- Beispiele
  - String
  - List
  - Arrays
  - eigene Klassen
  - ...

Die String Klasse

- ein String Objekt enthält eine Zeichenkette in festgelegter Reihenfolge
- die Klasse String verfügt über Methoden, die auf dem Objekt selber aufgerufen werden können

```
1    String msg = "dies␣ist␣ein␣String!";
2    String msg2 = msg.toUpperCase();
```

Arrays

Was ist ein Array?

- speichert eine genaue Anzahl an Variablen desselben Datentypes
- hat eine feste Größe (nicht dynamisch erweiterbar)
- nicht initialiserte Felder kriegen einen Standardwert oder null zugewiesen
- indices für schnellen Zugriff
- wird von Java wie ein Objekt behandelt → liegt auf dem Heap

```
1    int[] array1 = new int[5];
2    String[] array2 = {"A","R","R","A","Y"};
3    System.out.println(array2[0]); // gibt "A" aus
4    array1[3] = 42; // array1 = [0,0,0,42,0]
```

Mehrdimenionale Arrays

Arrays können auch mehrdimensional sein. Das bedeutet, dass wir Arrays von Arrays haben können. Ein zweidimensionales Array könnte beispielsweise so aussehen:

```
1    int[][] matrix = {{1, 2},{3, 4},{5, 6}};
2    System.out.println(matrix[2][0]); // gibt 5 aus
```

Gliederung

1. Organisation
2. Basics
   - Variablen
   - Datentypen
   - Arrays
3. Kontrollstrukturen
   - Verzweigungen
   - Schleifen
   - Idiomatisch
4. Methoden
   - Methodenaufruf
   - Main Methode
   - Rekursion
5. Classpath und Package-Pfad

Verzweigung

```
1    if(x){                          //kann aleine genutzt werden
2        //do X
3    } else if(y) {                  //falls spezieller Fall y
4        //do Y
5    } else {                        //default Fall
6        //do Z
7    }
```

- falls viele *else if's* auftreten sollten wir eher ein *switch statement* verwenden

Schleifen - While

```
1    //while Schleife
2    while(x){                    //solange die Bedingung x zutrifft
3        //do X
4    }
5    //do-while Schleife
6    do{
7    } while(x)
```

- Achtung vor Endlosschleifen!

Schleifen - *for*

```
for(int i = 0; i < 5; i++){
    //do X
}
```

- häufig benutzt, um Collections durchzugehen
- *for-each* Schleife:

```
for (int itVar : array){
    //do X
}
```

- hierbei bitte aufpassen. Mit der *for-each* Schleife kann nicht der Inhalt des Arrays geändert werden. Dies hat Performance Gründe

Logische Operatoren

Heinrich Heine Universität Düsseldorf

Logische Operatoren werden in Schleifenköpfen benötigt. Die Bedingung dafür, wie lange die Schleife läuft, ist also ein komplexer logischer Operator:

- `||`
- `&&`
- `==`
- < oder >
- <= oder >=
- `|`
- `&`

```
1    if(v + 42 <= 69){
2        // do X
3    }
```

Idiomatische Schleifen auswählen

Heinrich Heine Universität Düsseldorf

- For Loop -> falls wir den index benutzen
- Enhanced For Loop -> falls eine Datenstruktur komplett durchgegangen werden soll
- While Loop -> falls wir nicht wissen wie häufig wir etwas wiederholen müssen bspw. lesen einer Eingabe

Idiomatisch?

- Ist folgender Code idiomatisch?

```
public static void printList(List<Integer> list) {
    for (int i = 0; i < list.size(); i++) {
        System.out.print(list.get(i) + "␣");
    }
    System.out.println();
  }
```

- Sollten wir diesen ändern?

Idiomatisch?

- Ist folgender Code idiomatisch?

```
    public static int sumArray(List<Integer> list) {
        int count = 0;
        int summe = 0;
        while (count < list.size()) {
            summe += list.get(count);
            count++;
        }
        return summe;
  }
```

- Sollten wir diesen ändern?

Idiomatisch?

- Ist folgender Code idiomatisch?

```
    public static boolean contains(List<String> list, String toFind
        ) {
        boolean gefunden = false;
        for (String element : list) {
            if (element.equals(toFind)) {
                gefunden = true;
                break;
            }
        }
        return gefunden;
}
```

- Sollten wir diesen ändern?

Gliederung

1. Organisation
2. Basics
   - Variablen
   - Datentypen
   - Arrays
3. Kontrollstrukturen
   - Verzweigungen
   - Schleifen
   - Idiomatisch
4. Methoden
   - Methodenaufruf
   - Main Methode
   - Rekursion
5. Classpath und Package-Pfad

Methoden

Warum überhaupt Methoden?

- einfacher um sinnvolle Struktur zu erhalten
- Code wird wiederverwendbar
- wir können einfach spezifische Schnittstellen freigeben

Beispiel für eine Methode:

```
1    public static int sum(int a, int b){
2        int result = a + b;
3        return result;
4    }
```

Methoden Aufbau: Kopf

Heinrich Heine Universität Düsseldorf

Eine Methode besteht aus Methodenkopf und Methodenrumpf Der Methodenkopf ist wiefolgt aufgebaut:

```
//1.     2.     3.  4.  5.
  public static int sum(int a, int b){
      int result = a + b;
      return result;
  }
```

- 1. Sichtbarkeit: Ohne keyword package-private. Standard der Vorlesung
- 2. Kontext
- 3. Rückgabetyp
- 4. Name
- 5. Parameter

Methoden Aufbau: Rumpf

Heinrich Heine Universität Düsseldorf

Der Methodenrumpf beinhaltet die Logik der Methode. Wenn der Rückgabewert nicht void ist, dann muss an jedem Ende der Methode ein return stehen, welches den richtigen Datentypen zurück gibt. Folgendes wird also NICHT kompilieren:

```
1    static int isGreater(int a, int b){
2        if(a > b){
3            System.out.println(a);
4        } else {
5            return b;
6        }
7    }
```

Methodenaufruf

Heinrich Heine Universität Düsseldorf

Um eine Methode aufzurufen brauchen wir den Namen der Methode und müssen, falls die Methode Parameter benötigt diese in der korrekten Reihenfolge übergeben.

```
int a = 42;
int b = 3421
int summe = sum(a,b); //Wert der Variable summe = 3463
```

Hierbei müssen wir aber auch aufpassen, dass wir den Rückgabewert im richtigen Datentypen speichern. Folgendes funktioniert:

```
Integer summe = sum(a,b);
```

[1] Funktioniert nicht:

```
String summe = sum(a,b);
```

[1] Warum eigentlich Integer? int $\neq$ Integer. Oder nicht?

Main Methode

Die Main Methode hat als Parameter ein *String* Array. Der Inhalt des Arrays muss beim Aufruf des Programmes mit übergeben werden.

```
    static void main (String[] args){
        String msg = args[0];
    }
```

Falls keine Argumente übergeben werden

```
    static void main (){
        System.out.println("Hallo␣Welt!");
    }
```

Rekursion

Heinrich Heine Universität Düsseldorf

Was ist Rekursion?

- Methode, die sich selbt aufruft
- benötigt Rekursionsanker
- wenig code, jedoch häufig verwirrend

```
1    static void recursiveCount(int n){
2        System.out.println(n);
3        if( n <= 0){
4            return;
5        }
6        recursiveCount(n-1);
7    }
```

Gliederung

1. Organisation
2. Basics
   - Variablen
   - Datentypen
   - Arrays
3. Kontrollstrukturen
   - Verzweigungen
   - Schleifen
   - Idiomatisch
4. Methoden
   - Methodenaufruf
   - Main Methode
   - Rekursion
5. Classpath und Package-Pfad

Classpath und Package-Pfad

Heinrich Heine Universität Düsseldorf

Wir haben die java-Datei

`/home/marlin/progra/de/hhu/progra/Student.java`

mit folgendem Anfang:

Student.java

```java
package de.hhu.progra;

public class Student {
```

Welcher Teil vom Dateipfad ist der Classpath, welcher der Package-Pfad?

Classpath und Package-Pfad

Wir haben die java-Datei

`/home/marlin/progra` / `de/hhu/progra` `/Student.java`

mit folgendem Anfang:

Student.java

```java
package de.hhu.progra;

public class Student {
```

Welcher Teil vom Dateipfad ist der Classpath, welcher der Package-Pfad?

Classpath und Package-Pfad

Wir haben die java-Datei

`/home/marlin/progra` / `de/hhu/progra` `/Student.java`

mit folgendem Anfang:

Student.java

```java
package de.hhu.progra;

public class Student {
```

Welcher Teil vom Dateipfad ist der Classpath, welcher der Package-Pfad?

Was muss ich beachten, wenn ich in der Klasse `de.hhu.Verwaltung` die Klasse `de.hhu.progra.Student` verwenden möchte?